MUXQ: Mixed-to-Uniform Precision MatriX Quantization via Low-Rank Outlier Decomposition

Seoungsub Lee*, In Seo Kim, and Seon Wook Kim
School of Electrical Engineering, Korea University, Seoul, Korea
{tmdtjq789@korea.ac.kr, kiminse123@korea.ac.kr, seon@korea.ac.kr
*First author

April 6, 2026

Abstract

Large language models (LLMs) have achieved outstanding performance across a wide range of natural language processing tasks, but their enormous parameter counts impose substantial memory and computational overheads. This challenge is particularly critical in NPU-based on-device environments, where FP16/FP32 computation is inefficient and integer (INT) quantization is therefore essential. However, existing methods, including ZeroQuant, LLM.int8(), and SmoothQuant, do not fully address input-activation outliers and the associated hardware inefficiencies.

To overcome these limitations, we propose MUXQ (Mixed-to-Uniform Quantization). MUXQ detects outlier channels in input activations and introduces a small auxiliary matrix that redistributes outlier magnitudes across channels, thereby alleviating the outlier problem. This enables even activation outliers to be quantized at low-precision INT levels while preserving a hardware-friendly computation structure. Experiments on GPT-2 models at three scales (0.1B, 0.3B, and 0.7B parameters) using the WikiText-2 dataset show that MUXQ consistently achieves lower perplexity than naive quantization. In particular, under per-tensor quantization, MUXQ quantizes both activations and weights to INT8 while maintaining accuracy close to that of FP16. With only modest computational overhead, MUXQ enables stable low-precision inference and can be readily combined with other quantization techniques. These results suggest that MUXQ provides a promising direction for efficient and accurate LLM inference on edge devices.
https://github.com/GillchLee/MUXQ

1 Introduction

Large language models (LLMs) have recently demonstrated remarkable performance not only in natural language processing, but also in a wide range of applications such as question answering and summarization. As the demand for LLMs continues to grow, users increasingly require models that are both fast and accurate. Transformer-based LLMs typically rely on billions of parameters to achieve high accuracy during training and inference. However, when real-time inference is required, the number of parameters cannot be increased indefinitely for the sake of accuracy alone [1]. In addition, the growth in model size has steadily increased the memory usage of hardware accelerators such as graphics processing units (GPUs) [2], making real-time inference even more challenging in on-device environments.

Recently, the use of AI on edge devices, including smartphones and embedded systems, has expanded rapidly. Accordingly, the importance of techniques that can compress LLMs originally executed with floating-point arithmetic (FP16 and FP32) and enable efficient inference using integer (INT) operations has grown significantly [3]. This is particularly important in the case of neural processing units (NPUs), which support on-device AI acceleration and are especially optimized for INT computation. In such environments, directly implementing floating-point computation is not only inefficient, but can also impose severe limitations on inference.

Memory bottlenecks have also emerged as one of the major challenges in current LLM architectures. In particular, as model size increases, the KV cache can occupy more than half of GPU memory, which makes practical deployment increasingly difficult [2]. Therefore, it has become essential to replace floating-point computation with quantization techniques that compress

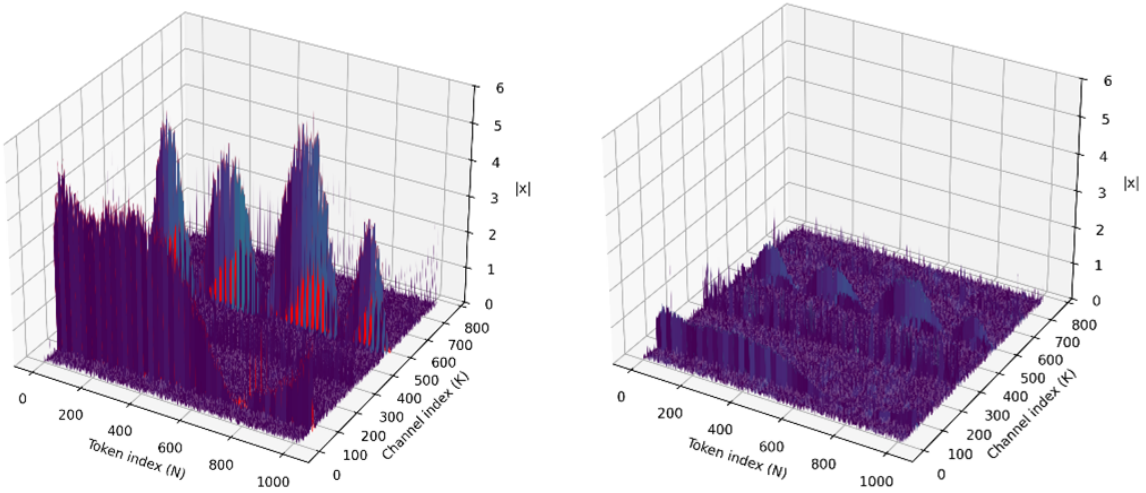

Figure 1: Figure 1. (Left) Activation outliers are concentrated in a small number of channels. (Right) After applying MUXQ, the magnitudes of the outlier channels are reduced.

models into low-precision integer formats, such as INT4 and INT8, in order to alleviate memory bottlenecks and enable efficient inference. Quantization is a technique that reduces memory usage and improves computational efficiency by converting model weights and activations into lower-precision representations [4]. For example, when FP16 is quantized to INT8, memory usage can be reduced by half, while general matrix multiplication (GEMM) can theoretically be accelerated by more than 2×. However, outliers in activation matrices make such quantization difficult. As shown in Fig. 1, these outliers are concentrated along only a few specific channels. If they are structurally removed or ignored, model accuracy degrades severely [5].

To address this problem, various quantization methods have been proposed, including LLM.int8() [6] and SmoothQuant [5]. LLM.int8() prevents significant accuracy degradation by separating outlier features and processing them in FP16 through mixed-precision decomposition. However, the remaining FP16 operations introduce irregular memory access patterns on hardware, which reduce hardware efficiency, and mixed-precision execution itself also lowers computational efficiency. SmoothQuant, on the other hand, reduces the difficulty of quantization by migrating the scaling factors of activations into the weights, but it does not fully resolve the fact that activation outliers still remain much larger than normal elements.

In this way, prior studies have attempted to minimize accuracy loss through mixed-precision computation or scaling-based mitigation. Nevertheless, they either fail to preserve hardware efficiency or leave activation outliers insufficiently addressed. Therefore, in LLM inference, detecting and handling activation outliers is a more critical issue for preserving performance than weight quantization, which is already relatively amenable to low-precision quantization.

In this paper, we propose MUXQ (Mixed-to-Uniform Precision Matrix Quantization) to fundamentally address this issue. MUXQ is a new quantization framework that removes the hardware bottlenecks of conventional mixed-precision schemes (FP16+INT8) and enables uniform low-precision INT8 quantization. Specifically, MUXQ detects activation outliers and adds a small auxiliary matrix to the input activation matrix of each layer. This auxiliary matrix splits and redistributes the magnitudes of the outlier channels, thereby alleviating extreme activation values. As a result, the original activation matrix becomes easier to quantize at low precision, and the overall activation distribution becomes more balanced, allowing high accuracy to be preserved using only INT precision. The right side of Fig. 1 illustrates how MUXQ produces a more balanced activation distribution.

Because all computations in MUXQ are performed in INT precision, it operates in a hardware-friendly manner on accelerators such as NPUs and eliminates the problem of irregular memory access. Although MUXQ introduces a small amount of additional memory usage and computational overhead, it enables stable inference at lower bit precision, such as INT8 or INT4, while reducing both overall system latency and memory usage. In our experiments, MUXQ achieved accuracy comparable to that of mixed-precision LLM.int8() on

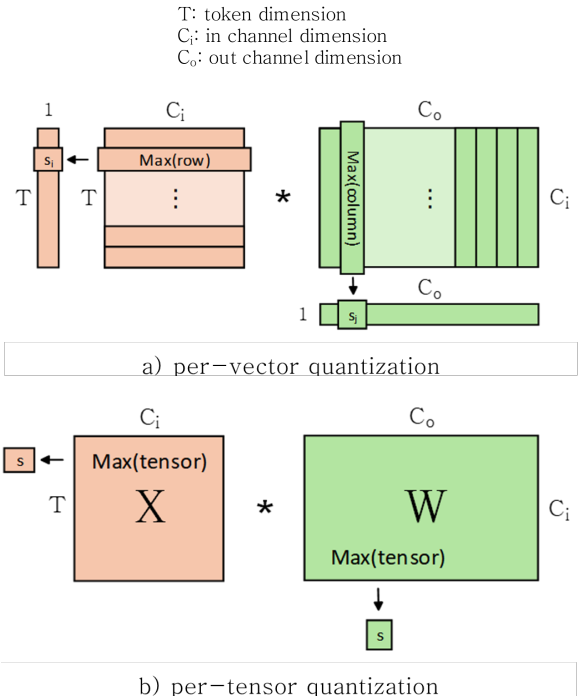

Figure 2: Quantization process for (a) per-vector quantization and (b) per-tensor quantization. In the per-vector case, activations and weights are quantized on a per-row or per-channel basis, respectively, and the scaling factor s_i is determined by the maximum value of each corresponding vector.

the 0.1B GPT-2 model, while successfully enabling uniform per-tensor INT8 quantization.

The contributions of this paper are as follows. First, we propose a new low-precision quantization approach that introduces an auxiliary matrix into the activation matrix to mitigate activation outliers. Second, MUXQ can be integrated with other quantization techniques and used in combination with them. For example, the activation matrix generated by MUXQ can further incorporate the difficulty-migration strategy of SmoothQuant to achieve additional performance improvement.

The remainder of this paper is organized as follows. Section II describes the background of quantization and the role of outliers in quantization. Section III presents the MUXQ framework and explains how it reduces the influence of outliers. Section IV compares MUXQ, naive quantization, and LLM.int8() on GPT-2 models under different quantization settings. Finally, Section V concludes the paper and discusses the significance and future potential of MUXQ.

2 Background

2.1 Quantization

Quantization refers to the process of mapping values represented in floating-point (FP) format to integer (INT) levels. Since INT operations are generally more efficient than FP operations on most hardware platforms, hardware efficiency can be improved by quantizing values, performing computation in INT, and then dequantizing the results afterward [4].

A basic quantization method is abs-max quantization, which quantizes values based on the element with the largest absolute magnitude within a given quantization unit. The following equations describe abs-max quantization. Specifically, the maximum absolute value within the quantization range is used to determine the scaling factor. The FP values are divided by this scaling factor and rounded to integers, and after integer matrix multiplication, the result is dequantized by multiplying the scaling factors back.

$$\begin{aligned} \bar{X}_{\text{INT8}} &= \text{round}\left(\frac{X_{\text{FP16}}}{s_X}\right), \\ s_X &= \frac{\max(|X_{\text{FP16}}|)}{2^7 - 1} \end{aligned} \tag{1}$$

$$\begin{aligned} \bar{W}_{\text{INT8}} &= \text{round}\left(\frac{W_{\text{FP16}}}{s_W}\right), \\ s_W &= \frac{\max(|W_{\text{FP16}}|)}{2^7 - 1} \end{aligned} \tag{2}$$

$$\bar{Y}_{\text{FP16}} = s_X \cdot s_W \cdot \left(\bar{X}_{\text{INT8}} \cdot \bar{W}_{\text{INT8}}\right) \tag{3}$$

The quantization unit determines the granularity, depending on the range over which the scaling factor is shared. Granularity can be categorized into per-group, per-vector, and per-tensor quantization. In the per-vector setting, the quantization unit may correspond either to a token vector or a channel vector, which are referred to as per-token and per-channel quantization, respectively. Because per-group quantization incurs excessive overhead [7], the experiments in this paper consider only per-vector and per-tensor granularity. Figure 2 illustrates the quantization schemes under different granularity settings.

2.2 Outlier handling

Outliers have abnormally large magnitudes compared with other elements, which makes the scaling factor excessively small. Figure 3 illustrates how the presence of outliers prevents values from being distributed evenly. Therefore, if only the magnitudes of the outlier channels can be reduced

to a level similar to that of normal channels, low-precision per-vector and per-tensor quantization can be enabled while preserving accuracy. Previous studies, as shown in Fig. 1, have observed that outlier features appear in a channel-wise manner. This suggests that quantizing activations on a per-channel basis could yield substantially higher accuracy [8]. However, per-channel activation quantization is difficult to support efficiently in GEMM kernels on hardware accelerators. For this reason, most prior works have adopted per-token or per-tensor activation quantization instead [5], [6]. Taking this practical constraint into account, this work also adopts per-token and per-tensor granularity for activation quantization.

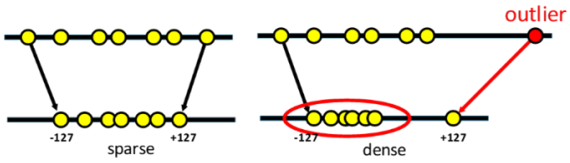

Figure 3: Figure 3. The presence of outliers affects the scaling factor, causing the value distribution to become denser and thereby increasing quantization error.

3 MUXQ structure

Given the channel-wise nature of activation outliers and their excessively large magnitudes, reducing the magnitudes of outlier channels is one promising way to address this problem. Based on this observation, we propose MUXQ (Mixed-to-Uniform Precision Matrix Quantization), which enables per-tensor quantization by redistributing the magnitudes of outlier channels through the addition of an auxiliary matrix to the activation matrix.

3.1 Motivation

In LLM fine-tuning, a widely used approach is to add a low-rank matrix to the weights and train only the added parameters [9]. This method effectively reduces fine-tuning cost and remains popular in practice. Inspired by this idea, MUXQ was developed from the intuition that low-precision quantization may become feasible if a low-rank auxiliary matrix is added not to the weights but to the activations, thereby redistributing the magnitudes of activation outlier channels.

3.2 Reducing Outlier Magnitudes

A floating-point number consists of a sign, an exponent, and a mantissa. Among these components, the exponent has the greatest influence on magnitude. Therefore, if the exponent can be adjusted appropriately so that outlier channels have magnitudes comparable to those of normal channels, quantization error can be significantly reduced compared with naive quantization. However, applying such a correction to outlier channels would require an additional operation immediately after matrix multiplication, which is difficult to implement efficiently on hardware such as systolic arrays and MAC units. Rather than applying a post-multiplication correction to the outlier channels, hardware efficiency can be improved by adding a low-rank matrix and compensating for the outliers through a separately computed correction term.

3.3 Outlier Decomposition method

In MUXQ, after applying the proposed decomposition, we refer to the resulting main matrix and auxiliary low-rank matrix as *Body* and *Aux*, respectively. Here, $\text{Body}_{\text{outlier}}$ denotes the submatrix formed by the outlier columns of the Body matrix. Through this decomposition, the magnitude of each outlier column is no longer concentrated in a single activation column, but is instead distributed across the Body and Aux terms.

Let X_{outlier} denote the submatrix formed by the outlier columns of the input activation matrix X. MUXQ decomposes this outlier submatrix into two parts: a reduced main component and an auxiliary component. Specifically,

$$\text{Body}_{\text{outlier}} = X_{\text{outlier}} \gg \exp \tag{4}$$

$$\text{Aux} = \text{Body}_{\text{outlier}} \tag{5}$$

and the original outlier contribution is reconstructed as

$$X_{\text{outlier}} = \text{Body}_{\text{outlier}} + (2^{\exp} - 1)\text{Aux}. \tag{6}$$

In this way, MUXQ reduces the dynamic range of the main activation path while preserving the original numerical contribution of the outlier columns through the auxiliary path.

An important design issue in MUXQ is the choice of $\exp factor$, which determine how aggressively the outlier magnitude is redistributed. As noted earlier, MUXQ adopts the outlier criterion used in LLM.int8(), where an outlier channel is defined as a channel containing at least one element with magnitude greater than 6. Under this criterion, we set $\exp factor = 2$ so that the outlier

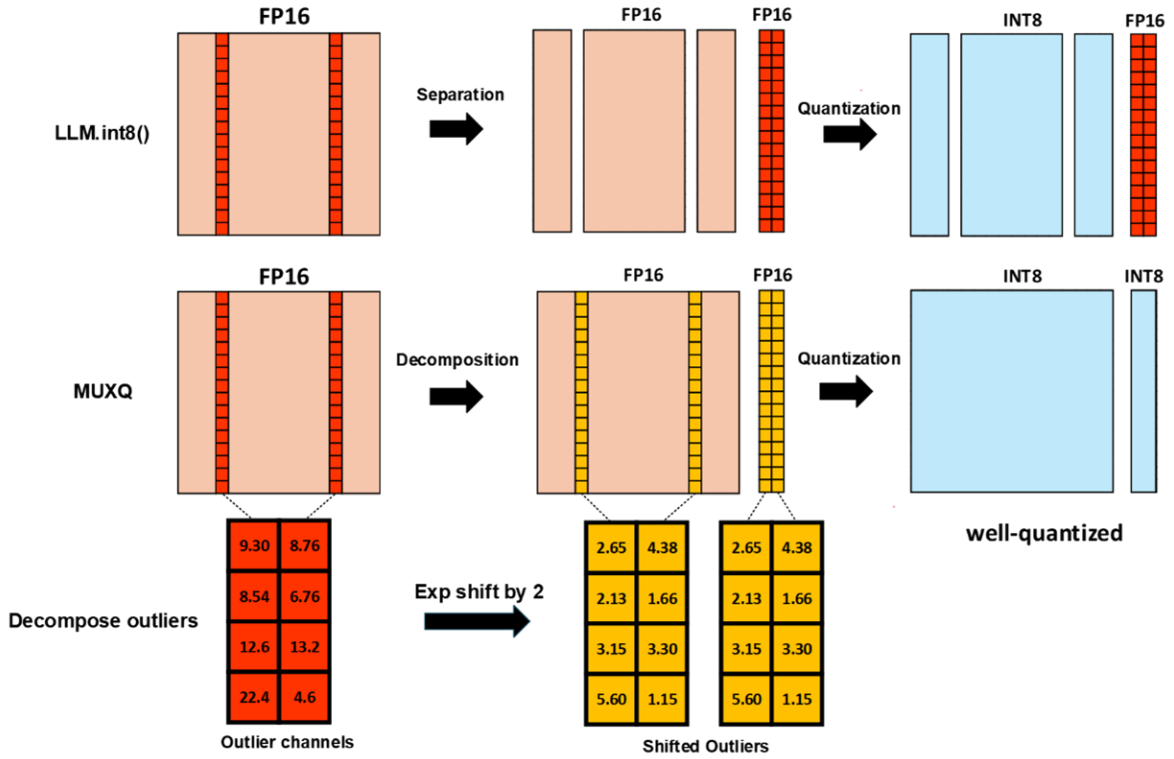

Figure 4: Comparison between the MUXQ architecture and the LLM.int8() architecture. The lower part of the figure illustrates an example of outlier decomposition in MUXQ under the assumption that exp $factor = 2$.

magnitude is reduced to a level comparable to that of normal channels. However, this choice involves an implementation trade-off. As can be seen from the equations below,

$$\begin{aligned} Y_{\text{body}} &= \text{Body} \cdot W, \\ Y_{\text{Aux}} &= \text{Aux} \cdot W, \\ Y_{\text{total}} &= Y_{\text{body}} + (2^{\exp} - 1)\, Y_{\text{Aux}} \end{aligned} \tag{7}$$

when exp $factor = 1$, MUXQ can be implemented simply by summing the outputs of two matrix multiplications. In contrast, when exp $factor$ are not equal to 1, additional overhead may be introduced. Therefore, exp $factor$ should be selected by considering both quantization effectiveness and implementation cost.

4 Experimental Setup

4.1 Evaluation method

In this section, we construct an experimental setup to evaluate the effectiveness of the proposed MUXQ. For comparison, we evaluate naive quantization, MUXQ, and LLM.int8() under the same conditions. We progressively increase the difficulty of quantization and compare the resulting performance changes across methods. Model performance is evaluated using language modeling perplexity.

4.2 Model and dataset

We use the WikiText-2[10] dataset for evaluation. WikiText-2 is a widely used benchmark for language modeling based on Wikipedia articles, making it suitable for evaluating language modeling performance across different models. The evaluation is conducted on the GPT-2 family, including GPT-2 small (0.1B), medium (0.3B), and large (0.7B). All models are pretrained checkpoints provided through Hugging Face and implemented using the Hugging Face Transformers framework.

4.3 Implementation

To minimize implementation complexity, we adopt the abs-max scaling method. Since scaling method is not treated as a primary experimental variable in this study, the same scaling method is used for all models. To incorporate the MUXQ structure, the original Conv1D modules in the Hugging Face framework are replaced with customized Conv1D modules. Quantization is applied to

Table 1: Perplexity comparison under different quantization settings.

model	granularity	target layers	bits		perplexity			
			IA(bits)	W(bits)	naive	MUXQ	llm.int8()	fp16
GPT2-small(0.1B)	per-vector	attention mlp	8	8	25.3825	25.28	25.2493	25.188
			7	8	25.8291	25.487	25.3867	25.188
			6	8	30.2001	26.9466	26.1048	25.188
			5	8	152.1251	43.129	30.5337	25.188
	per-tensor		8	8	50.618	**29.5534**	28.4595	25.188
GPT2-medium(0.3B)	per-tensor		8	8	22.5081	20.146	19.6749	18.4742
			7	8	35.8196	22.3645	20.4703	18.4742
			6	8	801.9368	38.2443	25.009	18.4742
GPT2-large(0.7B)	per-tensor		8	8	17.0662	16.7404	16.7041	16.4541
			7	8	19.4031	17.2334	17.0956	16.4541
			6	8	40.0405	**19.2885**	18.7935	16.4541

the main projection modules in GPT-2, including the attention modules (c_attn and the attention c_proj) and the MLP modules (c_fc and the MLP c_proj). However, instead of executing a true quantize-compute-dequantize inference pipeline, we evaluate accuracy using fake quantization[11], which follows a quantize-dequantize-compute procedure. Although fake quantization provides accuracy trends similar to those of simulated quantized inference, it does not reflect actual inference speed. Therefore, this study does not include an experimental comparison of inference latency. All experiments are conducted on an NVIDIA GeForce RTX 3080 (12 GB).

4.4 Experimental Results

Table 1 reports the perplexity results on the WikiText-2 dataset for the three GPT-2 models. Overall, MUXQ yields lower perplexity than naive quantization in most cases, although it generally remains slightly worse than LLM.int8(). The benefit of MUXQ in handling outliers is most clearly observed when activation precision is reduced. By contrast, reducing weight precision does not significantly affect the relative behavior of the outlier-handling methods, while varying quantization granularity enables a more detailed analysis and further confirms that MUXQ consistently outperforms naive quantization.

Among these factors, reducing activation precision plays the most important role in this study. Since the main purpose of the experiment is to evaluate the handling of activation outliers, the difference between MUXQ and naive quantization becomes more evident as activation precision decreases. As shown in Table 1, for all three GPT-2 models, once activations are quantized below a certain bit width, naive quantization suffers severe accuracy degradation, whereas MUXQ and LLM.int8() remain relatively stable. In particular, for GPT-2 medium, when activation precision is reduced to 7 bits, the perplexity of naive quantization rises sharply to above 35, while MUXQ and LLM.int8() maintain perplexity in the range of approximately 20 to 22. However, when activation precision is further reduced to 6 bits, the perplexity of MUXQ becomes substantially worse than that of LLM.int8(). This suggests that, as activation precision is pushed to lower bit widths, the FP16-based outlier handling of LLM.int8() becomes increasingly advantageous.

Table 2 shows that reducing weight precision results in performance changes of a similar magnitude across all three methods. This suggests that weight precision is not a major factor in distinguishing the three methods in this study, since both MUXQ and LLM.int8() primarily target activation outliers. In addition, as quantization granularity becomes finer, the baseline performance of the model improves significantly, indicating that the model becomes more robust to quantization. This makes it easier to clearly observe the advantage of MUXQ when activation precision is reduced. For example, in the GPT-2 large results with 6-bit activation precision, MUXQ maintains performance comparable to that of LLM.int8(), whereas naive quantization exhibits severe degradation, with perplexity increasing to more than twice its original value.

Table 2: Perplexity comparison under different bit settings.

		perplexity			
IA(bits)	W(bits)	naive	MUXQ	llm.int8()	fp16
8	5	27.5149	27.3616	26.5359	25.188
8	4	41.6420	41.1245	35.4289	25.188

4.5 Discussion

The goal of quantized model inference is to achieve faster inference through quantization without significantly sacrificing accuracy. In this study, we show that MUXQ can preserve accuracy relative to other quantization methods, but we do not experimentally evaluate inference speed. Nevertheless, because INT8 matrix multiplication is generally more efficient than FP16 matrix multiplication, MUXQ may offer practical deployment advantages in INT-oriented hardware environments, although this should be validated through kernel-level latency and power measurements on real hardware.

5 Conclusion

In this work, we proposed MUXQ to alleviate the activation-outlier problem in low-precision quantization of large language models. While existing methods either rely on mixed-precision computation at the cost of hardware efficiency to preserve accuracy or fail to sufficiently address outlier channels, MUXQ redistributes outlier magnitudes within INT precision to produce a more balanced activation distribution. In doing so, it achieves a better balance between accuracy and hardware efficiency.

MUXQ introduces an auxiliary matrix into the activation matrix to mitigate the magnitudes of channels in which outliers are concentrated, thereby achieving better performance than naive quantization. In addition, because the MUXQ structure does not conflict with other quantization approaches, it can be readily combined with additional techniques such as SmoothQuant or other calibration-based methods.

In summary, although MUXQ introduces a small amount of additional computation, it maintains low overhead and provides stable LLM inference performance even under uniform INT quantization. Future work may extend MUXQ by combining it with other quantization methods or by validating its speed and power efficiency on real hardware accelerators, ultimately moving toward practical on-device INT-only LLM acceleration.

References

[1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.

[2] M. Kim, S. Hong, R. Ko, S. Choi, H. Lee, J. Kim, *et al.*, "Oaken: Fast and efficient LLM serving with online-offline hybrid KV cache quantization," in *Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2025, pp. 482–497.

[3] V. Sze, Y. H. Chen, T. J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.

[5] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, Jul. 2023, pp. 38087–38099.

[6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30318–30332, 2022.

[7] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, *et al.*, "Q-BERT: Hessian-based ultra-low-precision quantization of BERT," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 34, no. 5, pp. 8815–8821, Apr. 2020.

[8] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of efficient transformer quantization," *arXiv preprint arXiv:2109.12948*, 2021.

[9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[10] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *arXiv preprint arXiv:1609.07843*, 2016.

[11] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-Power Computer Vision*, Chapman and Hall/CRC, 2022, pp. 291–326.